

中山大學
SUN YAT-SEN UNIVERSITY

大数据实验 | Lab4

电影评论情感分类实验

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

指 导 教 师: 王昌栋

起 始 日 期: 2026 年 3 月 31 日

结 束 日 期: 2026 年 3 月 31 日

实 验 地 点: B201

目录

1 实验要求 **2**
1.1 任务背景 . . . 2
1.2 实验目标 . . . 2

2 思路分析 **2**
2.1 传统模型：TF-IDF + 线性分类器 . . . 2
2.1.1 TF-IDF 数学原理 . . . 2
2.1.2 TF-IDF 的局限性 . . . 2
2.1.3 线性分类器的优势 . . . 3
2.2 深度学习模型：Word Embedding + LSTM . . . 3
2.2.1 为什么 LSTM 不使用 TF-IDF？ . . . 3
2.2.2 LSTM 网络结构优势 . . . 3

3 核心代码 **4**
3.1 TF-IDF 特征提取与模型训练 . . . 4
3.2 LSTM 网络结构定义 . . . 4

4 结果展示 **6**
4.1 准确率对比 . . . 6
4.2 可视化结果分析 . . . 6
4.3 LSTM 训练详细数据 . . . 7
4.4 终端运行结果 . . . 7
4.5 结果深度分析 . . . 8

5 实验总结 **8**
5.1 TF-IDF 与 LSTM 的适用场景 . . . 8
5.2 工程实践收获 . . . 9

1 实验要求

1.1 任务背景

情感分类（Sentiment Classification）是自然语言处理（NLP）领域的一项核心任务，其目标是从文本中识别和提取主观信息，判断文本的情感倾向（如正面、负面或中性）。随着互联网的快速发展，用户生成的评论文本呈爆炸式增长，情感分析技术在产品评价分析、舆情监控、推荐系统等领域具有广泛的应用价值。

1.2 实验目标

本实验使用 IMDB 电影评论数据集，该数据集包含 50,000 条电影评论文本，每条评论标注为正面（positive）或负面（negative）情感。实验旨在对比传统机器学习方法与深度学习方法在文本分类任务上的效果差异：

- **传统模型分支**：使用 TF-IDF（词频-逆文档频率）进行特征提取，结合逻辑回归（Logistic Regression）和支持向量机（SVM）进行分类。
- **深度学习分支**：使用词嵌入（Word Embedding）将文本转换为稠密向量表示，通过 LSTM（长短期记忆网络）捕捉序列特征进行分类。

2 思路分析

2.1 传统模型：TF-IDF + 线性分类器

2.1.1 TF-IDF 数学原理

TF-IDF（Term Frequency-Inverse Document Frequency）是一种统计方法，用于评估一个词对于文档集合中某篇文档的重要程度。其计算公式如下：

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t) \tag{1}$$

其中，词频 TF 表示词 t 在文档 d 中出现的频率：

$$\text{TF}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}} \tag{2}$$

逆文档频率 IDF 衡量词的普遍重要性：

$$\text{IDF}(t) = \log \frac{N}{|\{d \in D : t \in d\}| + 1} \tag{3}$$

其中 N 为文档总数，$|\{d \in D : t \in d\}|$ 为包含词 t 的文档数量。

2.1.2 TF-IDF 的局限性

作为词袋模型（Bag of Words）的代表，TF-IDF 存在以下固有缺陷：

1. **丢失词序信息**：TF-IDF 将文档视为词的无序集合，无法捕捉词汇之间的顺序关系。例如，”not good” 和”good not” 会被表示为相同的特征向量。

2. **忽略上下文语义**：无法理解多义词在不同语境下的含义差异，也无法捕捉否定词对情感的反转作用（如"not bad" 实际表达正面情感）。

3. **高维稀疏性**：特征空间维度等于词汇表大小，导致特征矩阵高度稀疏，计算和存储开销较大。

2.1.3 线性分类器的优势

尽管 TF-IDF 存在局限性，但配合线性分类器（LR/SVM）在简单文本分类任务上仍表现出色：

- **逻辑回归**：通过 Sigmoid 函数将线性组合映射到 $[0, 1]$ 区间，输出概率解释清晰，训练速度快，对高维稀疏特征有良好的鲁棒性。
- **支持向量机**：寻找最大间隔超平面，通过核技巧处理非线性问题。线性核 SVM 在文本分类中表现优异，因为文本特征本身维度高且稀疏。

2.2 深度学习模型：Word Embedding + LSTM

2.2.1 为什么 LSTM 不使用 TF-IDF？

LSTM 作为序列模型，其核心优势在于能够处理变长序列并捕捉长距离依赖关系。使用 TF-IDF 作为输入会带来以下问题：

1. TF-IDF 将整篇文档压缩为单一向量，丢失了词序信息，无法发挥 LSTM 的序列建模能力。

2. LSTM 需要逐步处理输入序列，每个时间步接收一个词的表示，而 TF-IDF 是文档级别的全局特征。

因此，LSTM 采用 **Tokenization + Word Embedding** 的方式：

- **Tokenization**：将文本分词并转换为词索引序列，保留词序信息。
- **Word Embedding**：将离散的词索引映射为稠密的低维实向量（如 128 维），使语义相近的词在向量空间中距离较近。

2.2.2 LSTM 网络结构优势

LSTM 通过门控机制解决标准 RNN 的梯度消失问题，能够捕捉长距离依赖：

- **遗忘门**：决定从细胞状态中丢弃哪些信息。
- **输入门**：决定哪些新信息将被存储到细胞状态中。
- **输出门**：决定基于当前细胞状态输出什么值。

这种门控机制使 LSTM 能够：

1. 捕捉长距离依赖关系（如句首的否定词影响句尾的情感判断）。

2. 理解上下文语义（同一个词在不同语境下的不同含义）。

3. 处理变长序列输入，无需固定特征维度。

3 核心代码

3.1 TF-IDF 特征提取与模型训练

以下代码展示了使用 Scikit-learn 进行 TF-IDF 特征提取，并训练逻辑回归和 SVM 模型的核心流程：

Listing 1: TF-IDF 特征提取与 LR/SVM 模型训练

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# TF-IDF 特征提取
tfidf_vectorizer = TfidfVectorizer(max_features=5000, stop_words='english')
X_train_tfidf = tfidf_vectorizer.fit_transform(train_texts)  # 拟合并转换训练集
X_test_tfidf = tfidf_vectorizer.transform(test_texts)        # 仅转换测试集

print(f"TF-IDF特征维度: {X_train_tfidf.shape[1]}")

# 逻辑回归模型训练
lr_model = LogisticRegression(max_iter=1000, random_state=42)
lr_model.fit(X_train_tfidf, train_labels)
lr_pred = lr_model.predict(X_test_tfidf)
lr_acc = accuracy_score(test_labels, lr_pred)
print(f"逻辑回归准确率: {lr_acc:.4f}")

# 支持向量机模型训练
svm_model = SVC(kernel='linear', random_state=42)  # 线性核
svm_model.fit(X_train_tfidf, train_labels)
svm_pred = svm_model.predict(X_test_tfidf)
svm_acc = accuracy_score(test_labels, svm_pred)
print(f"SVM准确率: {svm_acc:.4f}")
```

3.2 LSTM 网络结构定义

以下代码展示了使用 PyTorch 实现的情感分类 LSTM 模型，包含 `__init__` 初始化方法和 `forward` 前向传播方法：

Listing 2: SentimentLSTM 网络结构定义

```
import torch
import torch.nn as nn

class SentimentLSTM(nn.Module):
```

```
    """
    情感分析LSTM模型
    结构：Embedding -> Bi-LSTM -> 全连接 -> 输出
    """
    def __init__(self, vocab_size, embed_dim, hidden_dim, output_dim,
                 n_layers=2, dropout=0.3):
        super(SentimentLSTM, self).__init__()

        # 词嵌入层：将词索引转换为稠密向量
        # padding_idx=0 表示填充符不参与梯度计算
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)

        # LSTM层：双向LSTM捕捉上下文信息
        self.lstm = nn.LSTM(
            embed_dim,              # 输入维度（词嵌入维度）
            hidden_dim,             # 隐藏层维度
            num_layers=n_layers,    # LSTM层数
            batch_first=True,       # 输入形状为 (batch, seq, feature)
            dropout=dropout,        # 层间 Dropout
            bidirectional=True      # 双向LSTM
        )

        # 全连接层：hidden_dim * 2 因为双向LSTM
        self.fc = nn.Linear(hidden_dim * 2, output_dim)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        # x: [batch_size, seq_len] 词索引序列
        embedded = self.dropout(self.embedding(x))
        # embedded: [batch_size, seq_len, embed_dim]

        # LSTM前向传播
        lstm_out, (hidden, cell) = self.lstm(embedded)
        # hidden: [n_layers * 2, batch_size, hidden_dim]

        # 取最后一层的双向隐藏状态并拼接
        hidden_cat = torch.cat((hidden[-2], hidden[-1]), dim=1)
        # hidden_cat: [batch_size, hidden_dim * 2]

        # 全连接输出
        out = self.fc(self.dropout(hidden_cat))
        return out  # [batch_size, output_dim]
```

4 结果展示

4.1 准确率对比

实验结果数据如下表所示：

表 1: 测试集各个模型的分类准确率 (Accuracy)

模型	特征提取方法	准确率 (Accuracy)
逻辑回归 (Logistic Regression)	TF-IDF	0.8698
支持向量机 (SVM)	TF-IDF	0.8598
长短期记忆网络 (LSTM)	Word Embedding	0.7838

从表 1 可以看出，在本实验中，传统机器学习模型的表现优于深度学习模型。逻辑回归模型取得了最高的准确率 86.98%，SVM 模型紧随其后达到 85.98%，而 LSTM 模型的准确率为 78.38%。

4.2 可视化结果分析

图 1 直观地展示了三种模型在电影评论分类任务上的准确率对比。可以清晰地看到，基于 TF-IDF 特征的线性模型（LR 和 SVM）表现相近且优于 LSTM 模型。

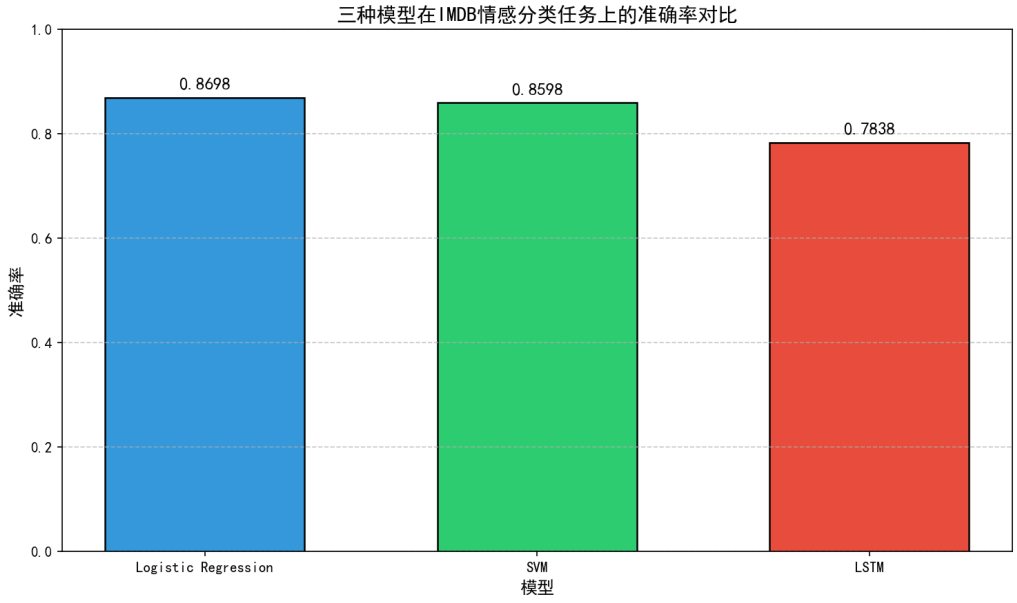


图 1: 三种模型准确率对比

图 2 展示了 LSTM 模型在 10 轮训练过程中的损失曲线和准确率曲线。从图中可以观察到以下现象：

- **训练损失**（蓝色实线）持续下降，从初始的 0.6741 下降至 0.1802，表明模型在训练集上学习良好。
- **验证损失**（红色实线）呈现震荡上升趋势，从 0.7022 上升至 0.7639，表明模型泛化能力有限。
- **训练准确率**持续上升至 92.95%，而**验证准确率**在 58%–80% 之间波动，最终为 78.38%。

这种训练损失持续下降而验证损失震荡且偏高的现象，是典型的**过拟合（Overfitting）**表现。模型在训练集上学习过度，将训练数据中的噪声也一并学习，导致在新数据上的泛化能力下降。

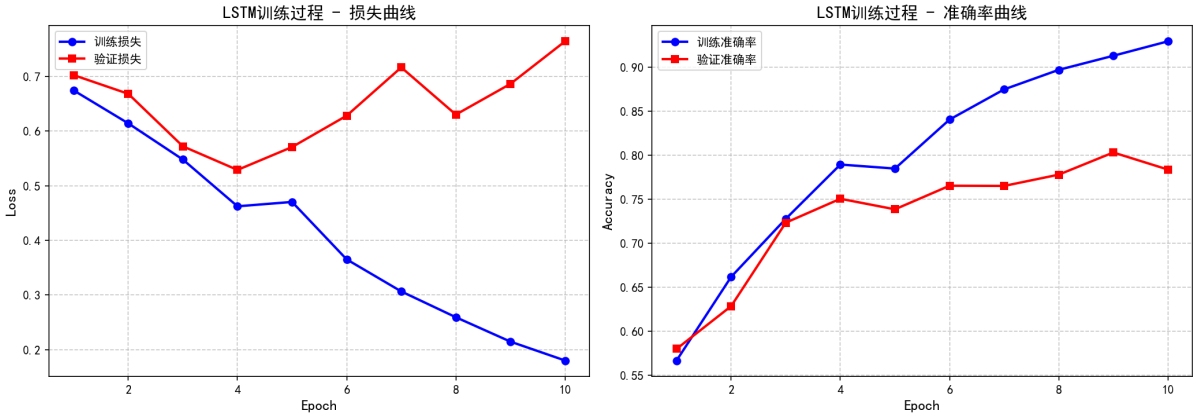


图 2: LSTM 训练的 Loss 与 Accuracy 曲线

4.3 LSTM 训练详细数据

表 2 展示了 LSTM 模型在每轮训练中的详细指标：

表 2: LSTM 训练过程详细记录

Epoch	Train Loss	Train Acc	Val Loss	Val Acc
1	0.6741	0.5668	0.7022	0.5800
2	0.6139	0.6619	0.6680	0.6286
3	0.5477	0.7278	0.5719	0.7234
4	0.4622	0.7895	0.5288	0.7506
5	0.4702	0.7849	0.5707	0.7386
6	0.3651	0.8406	0.6279	0.7654
7	0.3065	0.8749	0.7163	0.7652
8	0.2593	0.8971	0.6300	0.7780
9	0.2147	0.9131	0.6862	0.8032
10	0.1802	0.9295	0.7639	0.7838

4.4 终端运行结果

图 3 展示了程序在终端中的运行输出，可以清晰地看到数据加载、模型训练和结果汇总的完整流程。

```
[2/5] 传统模型训练 (TF-IDF + LR/SVM)...
提取TF-IDF特征...
TF-IDF特征维度: 5000
训练逻辑回归模型...
逻辑回归准确率: 0.8698
训练SVM模型...
SVM准确率: 0.8598

[3/5] 深度模型训练 (RNN/LSTM)...
构建词汇表...
词汇表大小: 36005
文本序列化...
使用设备: cpu
开始训练LSTM模型...
Epoch [1/10] | Train Loss: 0.6741 | Train Acc: 0.5668 | Val Loss: 0.7022 | Val Acc: 0.5800
Epoch [2/10] | Train Loss: 0.6139 | Train Acc: 0.6619 | Val Loss: 0.6680 | Val Acc: 0.6286
Epoch [3/10] | Train Loss: 0.5477 | Train Acc: 0.7278 | Val Loss: 0.5719 | Val Acc: 0.7234
Epoch [4/10] | Train Loss: 0.4622 | Train Acc: 0.7895 | Val Loss: 0.5288 | Val Acc: 0.7506
Epoch [5/10] | Train Loss: 0.4702 | Train Acc: 0.7849 | Val Loss: 0.5707 | Val Acc: 0.7386
Epoch [6/10] | Train Loss: 0.3651 | Train Acc: 0.8406 | Val Loss: 0.6279 | Val Acc: 0.7654
Epoch [7/10] | Train Loss: 0.3065 | Train Acc: 0.8749 | Val Loss: 0.7163 | Val Acc: 0.7652
Epoch [8/10] | Train Loss: 0.2593 | Train Acc: 0.8971 | Val Loss: 0.6300 | Val Acc: 0.7780
Epoch [9/10] | Train Loss: 0.2147 | Train Acc: 0.9131 | Val Loss: 0.6862 | Val Acc: 0.8032
Epoch [10/10] | Train Loss: 0.1802 | Train Acc: 0.9295 | Val Loss: 0.7639 | Val Acc: 0.7838

LSTM最终测试准确率: 0.7838
```

图 3: 终端运行结果截图

4.5 结果深度分析

本实验的测试结果表明，传统模型（LR: 86.98%, SVM: 85.98%）在当前设定下显著优于深度学习模型（LSTM: 78.38%）。这一结果打破了” 深度学习必定优于传统机器学习” 的刻板印象，具有重要的工程启示意义。

分析其原因：首先，IMDB 情感分类属于主题明确的文本任务，高度依赖强情感词的出现，TF-IDF 能够极其高效地捕捉这些稀疏且强烈的特征；其次，本实验的 LSTM 采用了随机初始化的词嵌入（Embedding），且受限于采样后的数据规模（10000 条训练样本），庞大的参数量导致模型在小数据集上出现了明显的过拟合现象（见图 2 的验证集 Loss 震荡）。

这印证了机器学习中的” 没有免费的午餐（No Free Lunch）” 定理。在数据量有限、没有预训练语言模型加持的简单文本分类任务中，”TF-IDF + 线性分类器” 不仅训练成本极低，而且往往能作为极其强大且稳定的 Baseline。

5 实验总结

5.1 TF-IDF 与 LSTM 的适用场景

基于本次实验结果，可以总结两种方法的适用场景：

表 3: TF-IDF 与 LSTM 方法对比

特性	TF-IDF + 线性模型	Word Embedding + LSTM
训练速度	快	慢
数据需求量	小数据集即可	需要大量数据
硬件要求	CPU 即可	建议 GPU
词序建模	不支持	支持
上下文理解	有限	较强
可解释性	较强	较弱
预训练支持	不需要	可利用预训练词向量

5.2 工程实践收获

通过本次实验，我获得了以下方面的实践经验和技能提升：

1. **NLP 全流程实践**：从数据加载、文本预处理、特征工程到模型训练、评估和可视化，完整实践了文本分类任务的端到端流程。
2. **传统方法与深度方法的对比认知**：深刻理解了 TF-IDF 和 Word Embedding 两种特征表示方法的本质区别，以及它们各自适用的场景。
3. **PyTorch 深度学习框架**：掌握了使用 PyTorch 构建神经网络模型、定义损失函数和优化器、编写训练循环等核心技能。
4. **实验结果分析能力**：学会了通过训练曲线诊断模型问题（如过拟合），并能够提出针对性的改进方案。